

Computer Vision:

Homework-9

Bilal Ahmed
ahmedb@purdue.edu

1 Theoretical Questions

Justify the claim: Given a pixel x in the left camera, we can be sure that its corresponding pixel x' in the right camera will be on the Epipolar Line l' shown in the Fig. 1.

Answer: Point $\vec{x}$ in the left camera is the projection of the world point $\vec{X}$ onto the left camera. If we think of line connecting the point $\vec{X}$ and center of left camera $\vec{C}$ it can be represented as;

$$\vec{l} = \vec{C} + \lambda \vec{X}$$

The projection of this line on right camera would be given by;

$$P'(\vec{l}) = P'(\vec{C}) + \lambda P'(\vec{X})$$

where P' is the projection matrix of the right camera. Since $P'(\vec{C})$ is the epipolar point in the right camera, the expression we got through projection of line $\vec{l}$ gives the epipolar lines (by definition). $P'(\vec{X})$ is the projection of the world point ($\vec{X}$) onto the right camera i.e. it is the pixel x' corresponding to the point x in the left camera. Hence that proves the claim that x' lies on the epipolar line.

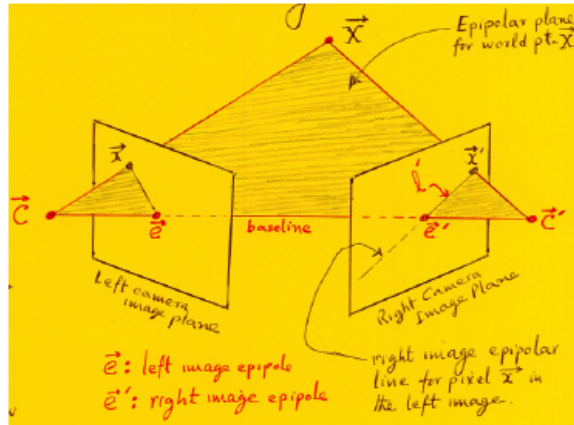


Figure 1: Epipolar Geometry

2 Projective Stereo Reconstruction

2.1 Image Rectification

The first step for projective stereo reconstruction is image rectification. Here are the steps involved;

1. Getting at least 8 manual correspondences between left and right views of images.
2. Normalize the point correspondences $\mathbf{x}$ and $\mathbf{x}'$ using linear transforms T and T' such that the normalized pixel coordinates have $(0,0)$ at the center of the images and the mean distance of pixels from center is $\sqrt{2}$
3. Compute the fundamental matrix F using the epipolar geometry relationship for every pixel $\mathbf{x}$ in left image and it's corresponding pixel $\mathbf{x}'$ in the right image.

$$\mathbf{x}F\mathbf{x}' = 0$$

If we write $\mathbf{x} = (x, y, 1)$ and $\mathbf{x}' = (x', y', 1)$, then the above eq. is equivalent to:

$$\begin{bmatrix} xx' & x'y' & x' & xy' & yy' & y' & x & y & 1 \end{bmatrix}^T \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0.$$

Using at least 8-point correspondences we can get a linear equation as;

$$Af = 0$$

where A is the matrix of coefficients and f is the vector of elements of F . Since F is homogeneous, we can find linear least square solution to get the fundamental matrix F .

4. Since F must have rank-2, we can enforce this constraint using singular value decomposition and setting the smallest singular value to 0.
5. Using normalization matrices T and T' to get the estimate of F using

$$F = T'^{-1} \tilde{F} T$$

6. Once we get F , we can get $\mathbf{e}$ and $\mathbf{e}'$ as right and left null vector of F .
7. Using the canonical configuration we can use P as;

$$\mathbf{P} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

and P' as;

$$\mathbf{P}' = [[\mathbf{e}']_{\times} \mathbf{F} \mid \mathbf{e}']$$

where

$$[\mathbf{e}']_{\times} = \begin{bmatrix} 0 & -e'_3 & e'_2 \\ e'_3 & 0 & -e'_1 \\ -e'_2 & e'_1 & 0 \end{bmatrix}$$

8. We can use Levenberg-Marquardt to refine the estimate of P' by computing geometric loss in the space of pixels. For that we compute world coordinates X_i for every correspondence (x_i, x'_i) using triangulation and then project the world point X_i back to pixel space using the estimates of projection matrices P and P' . Here is the expression for geometric distance;

$$\sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 + \|\mathbf{x}'_i - \hat{\mathbf{x}}'_i\|^2$$

where $\hat{\mathbf{x}}_i$ and $\hat{\mathbf{x}}'_i$ are the projections of world coordinate X back to the left and right images.

9. We get the right homography matrix using

$$\mathbf{H}' = T_2 G R T_1$$

where

$$T_1 = \begin{bmatrix} 1 & 0 & -x_0 \\ 0 & 1 & -y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T_2 = \begin{bmatrix} 1 & 0 & x_0 \\ 0 & 1 & y_0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$G = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -\frac{1}{f} & 0 & 1 \end{bmatrix}$$

where $f = |(e'_1 - x_0) \cos \theta - (e'_2 - y_0) \sin \theta|$

$$R = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

where $\theta = \arctan2(e'_2, e'_1)$

10. For getting the left homography matrix using;

$$\mathbf{M} = \mathbf{P}' \mathbf{P}^+$$

$$\mathbf{H}_0 = \mathbf{H}' \mathbf{M}$$

$$\mathbf{H}_A = \begin{bmatrix} a & b & c \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Then, we compute $\mathbf{z}_i = (rx_i, ry_i, 1)$ and $\mathbf{z}'_i = (rx'_i, ry'_i, 1)$, where:

$$\mathbf{z}_i = \mathbf{H}_0 \mathbf{x}_i$$

$$\mathbf{z}'_i = \mathbf{H}' \mathbf{x}'_i$$

We can build the linear system:

$$\begin{bmatrix} rx_1 & ry_1 & 1 \\ \vdots & \vdots & \vdots \\ rx_N & ry_N & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} rx'_1 \\ \vdots \\ rx'_N \end{bmatrix}$$

to solve for a , b , and c using linear least squares. The left homography $\mathbf{H}$ is given by:

$$\mathbf{H} = \mathbf{H}_A \mathbf{H}_0$$

Rectified images are obtained using these matrices $\mathbf{H}$ and $\mathbf{H}'$, such rectified images are shown in Fig. 3

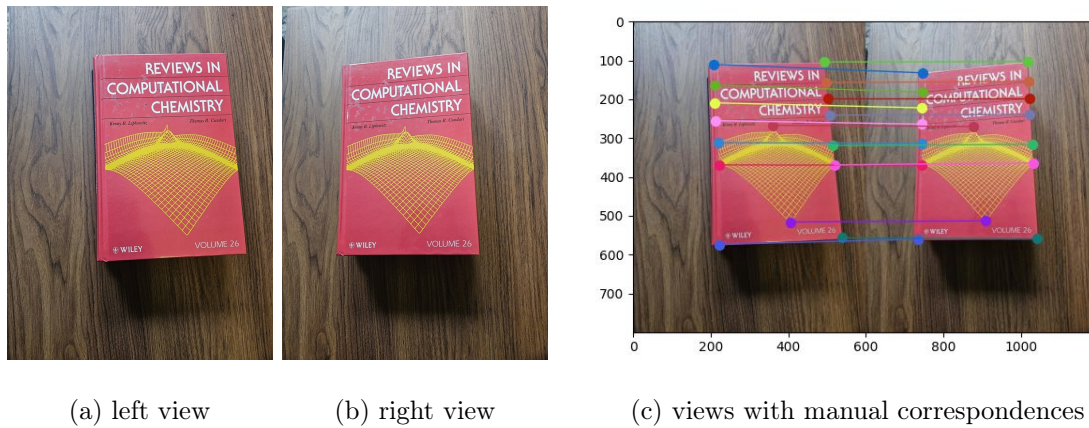


Figure 2: Two views of the image along with the manual correspondences between the views.

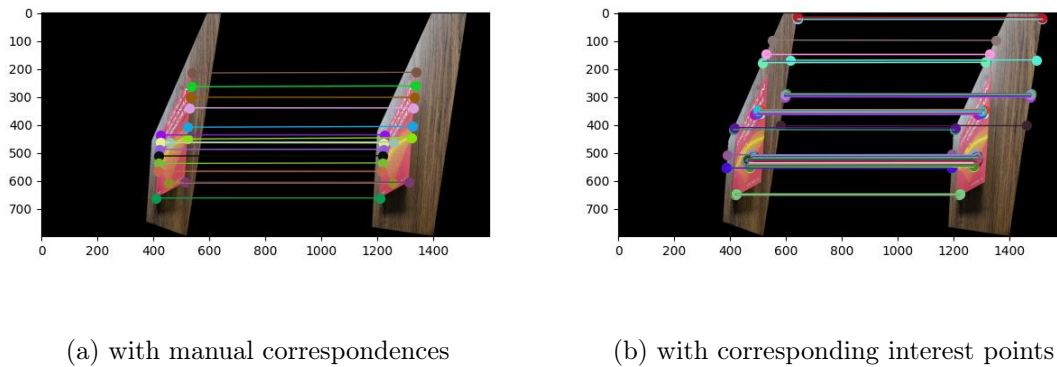


Figure 3: Rectified images with correspondences between views.

2.2 Interest Point Detection

We need more correspondences to get accurate estimate of world points. We can get these correspondences by extracting interest points using Canny edge detector. Here are the steps;

1. Get canny edge mask for rectified left and right images using gray scale images.
2. Since images are rectified, we can match the interest points by computing SSD between interest point in the left image and candidate points within the same row in the right image. Since the rectified images might not be perfect, I search for matching interest points in 3 rows, one row above and below the matching row.
3. Each key point in the left image is matched to exactly one key point in the right image based on SSD.

2.3 Projective Reconstruction

We use these key point pairs to get projections of 3D world points using Lavenburge-Marquardt to minimize the geometric distance mentioned above but only optimizing for world points and not for projection matrix P' . Fig.4 shows 2 views of 3D projections of world points, with lines connecting the world points to the corresponding key points in left and right images.

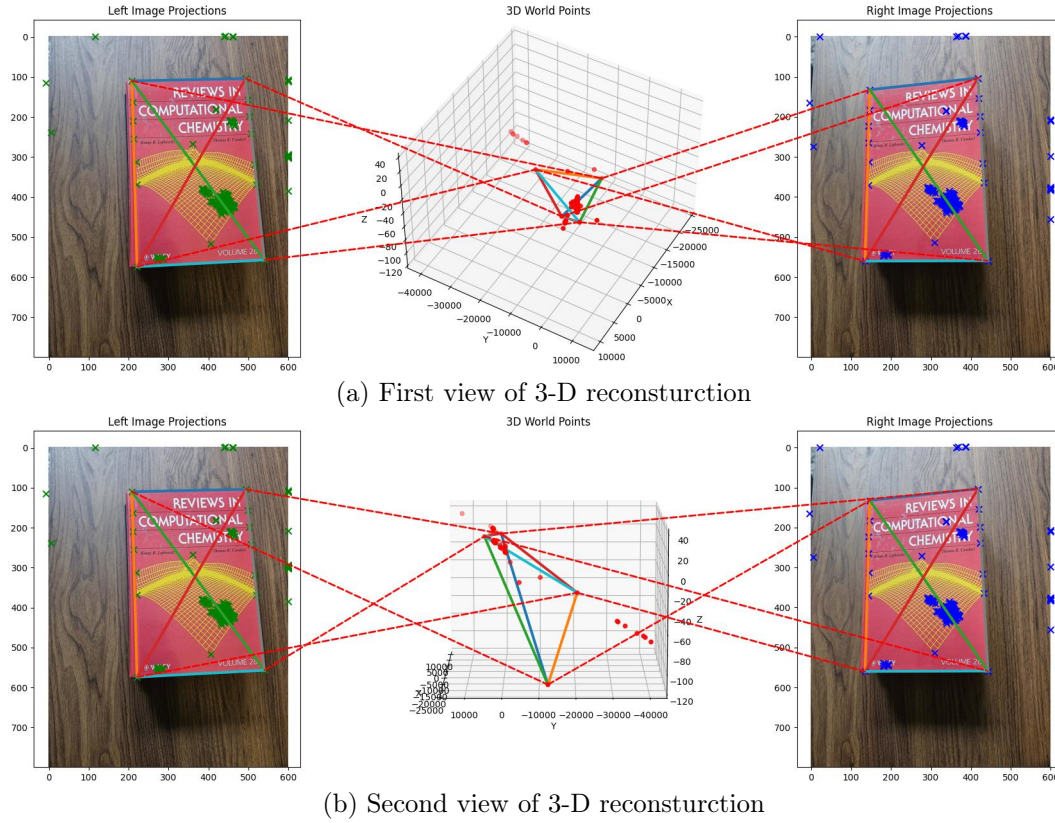


Figure 4: Visualizations of projected 3D World points with lines connecting corresponding points in left view, 3D world and right view.

3 Loop And Zhang

The Loop and Zhang algorithm decomposes the rectification homographies H and H' into distinct components as follows:

$$H = H_{sh}H_{sim}H_p$$

$$H' = H'_{sh}H'_{sim}H'_p$$

In this formulation, H and H' are purely projective homographies that serve to map the epipoles e and e' to infinity in their respective image planes. The homographies H_{sim} and H'_{sim} are similarity transformations that rotate the epipoles, which have been sent to infinity by the projective transformations, so that they align along the world-X axis. Finally, the shear homographies H_{sh} and H'_{sh} are designed to minimize projective distortion by adding extra degrees of freedom to the rectification process.

Fig.5 show the original views of the images with epipolar lines on top of the images. Fig.18 show the rectified images using Loop and Zhang algorithm.

Rectified images obtained by Loop and Zhang are different from the images rectified by my implementation by some projective transform. This could be due to the different interest points detected by Loop and Zhang compared to those detected by my algorithm based on Canny edge detector. My interest points detector finds some key points in the wooden background in the image. This could be because the background is not entirely uniform and it has certain texture so Canny edge detector finds some points in the background as well. Whereas the Loop And Zhang algorithm only detects interest points within the foreground object (the book).

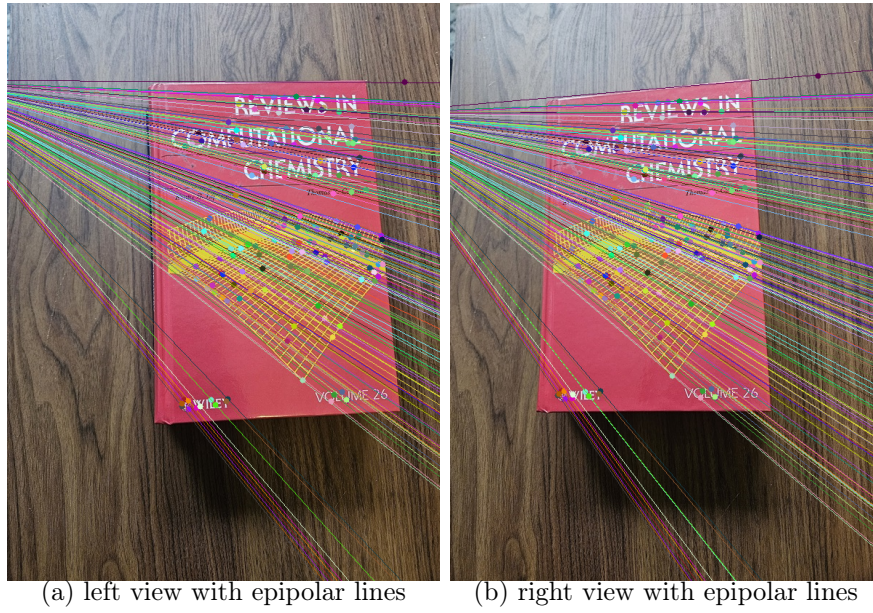


Figure 5: Left and right views along with epipolar lines (using LoopAndZhang)

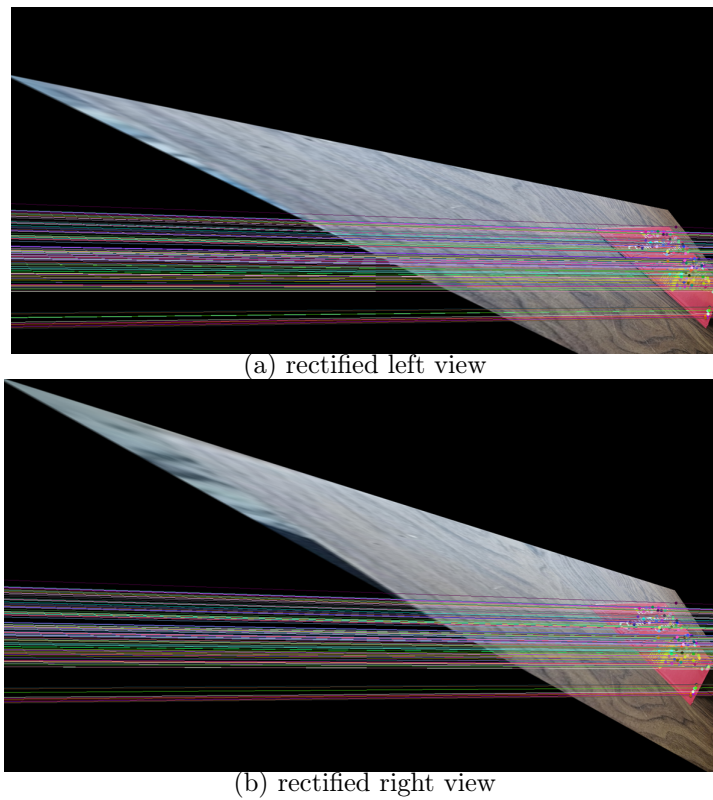


Figure 6: Rectified left and right views along with epipolar lines (using LoopAndZhang)

4 Dense Stereo Matching

Dense Stereo Matching is a complicated task, here we used census transform to compute it.

1. We compute the disparity map by computing the disparity of each pixel in the left image with

the possible candidates in the right image and assign disparity value based on the smallest disparity cost.

2. The ground truth disparity map is re-normalized as mentioned in the handout by dividing the disparity values by 4 to make them consistent with disparities at the resolution we are working with.
3. Maximum disparity value d_{max} is computed from the ground truth disparity map.
4. For each pixel in the left image, denoted (x_i, y_i) , we assign a candidate correspondence pair in the right image denoted $(x'_i - d, y'_i)$ where $d \in [0, \dots, d_{max})$
5. For each pixel we assign binary value of 1 to the neighbor whose pixel value is greater than the center and 0 otherwise. This the neighborhood around the pixel can be represented by bitvector.
6. For bitvector in left image, we compute cost of bitvector in left image with every candidate by computing XOR of bitvectors and counting the number of ones in the outcome.
7. We assign the disparity value as the value d giving the minimum cost among all the candidates in the right image.
8. Assigning disparity value at each pixel value gives us a disparity map.

Fig.7 and Fig.8 show disparity maps for two different window sizes as well as error maps of disparity map vs the ground truth.

4.1 Discussion

The results show that the accuracy is not that good for computing the disparity map indicating the challenge involved in the task. Using larger window size gives better accuracy compared to the smaller window, although the difference is not that huge. Here is the summary of accuracy using two different windows;

Table 1: Accuracy of disparity map with different window sizes.

window	accuracy
(9,9)	0.638
(11,11)	0.658

5 Automatic Extraction of Dense Correspondences using Depth Maps

Given the depth maps and calibration parameters of the cameras, we can use depth check to get the dense correspondences between pair of images. Using the calibration parameters we can project a pixel in the image as ray going outwards. Since we have additional depth information, we can get the location of world coordinates of the projected point. The world point is then projected onto the coordinates of second camera using extrinsic parameters, giving us the depth estimate. Then these coordinates in second camera coordinates are projected onto plane of the camera (using the intrinsic) and ground truth depth is obtained using the depth map, we calculate the difference in the estimated depth and the ground truth depth. If this difference is less than the threshold, the depth is matched and the correspondence is accepted.

These extracted dense correspondences are useful in many applications like stereo matching, 3D reconstruction etc. This is especially useful for training deep learning models based on key points extraction.

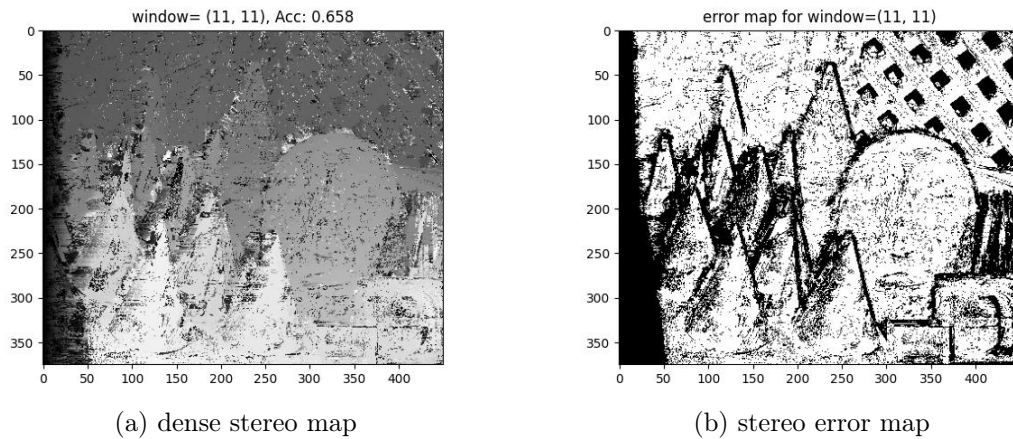


Figure 7: Dense stereo map and error map for window 1. Window size and accuracy are displayed in the title

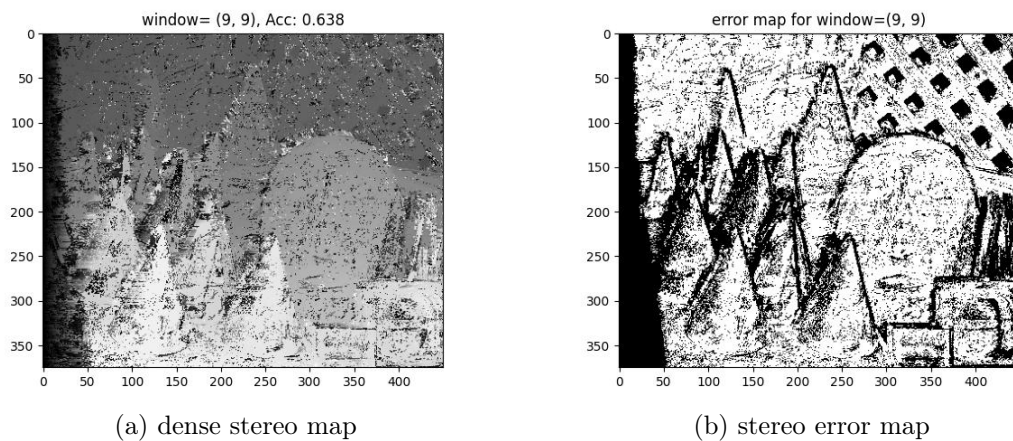


Figure 8: Dense stereo map and error map for window 2. Window size and accuracy are displayed in the title

Python Code

5.1 Code for Task-1 and 3

```

1  import cv2
2  import pathlib
3  # import skimage
4  import sklearn.cluster
5  import scipy
6  import computer_vision
7  import BitVector
8  import random
9  import numpy as np
10 import matplotlib.pyplot as plt
11 from mpl_toolkits.mplot3d import Axes3D
12 import cv2
13
14 import matplotlib.pyplot as plt

```

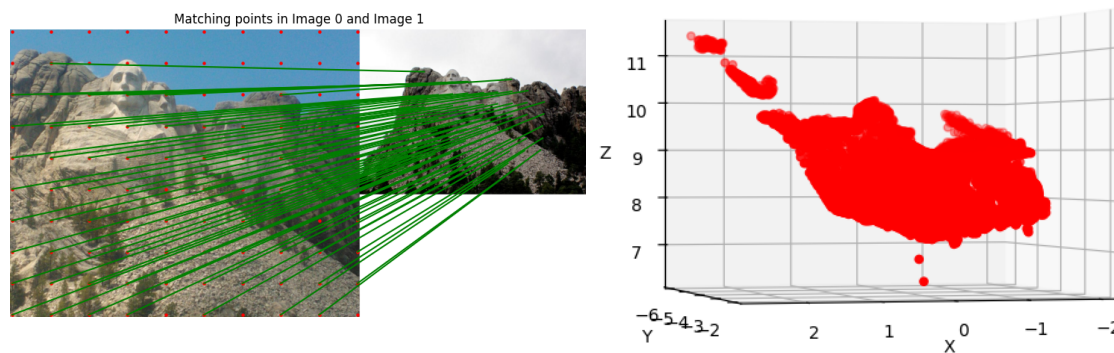
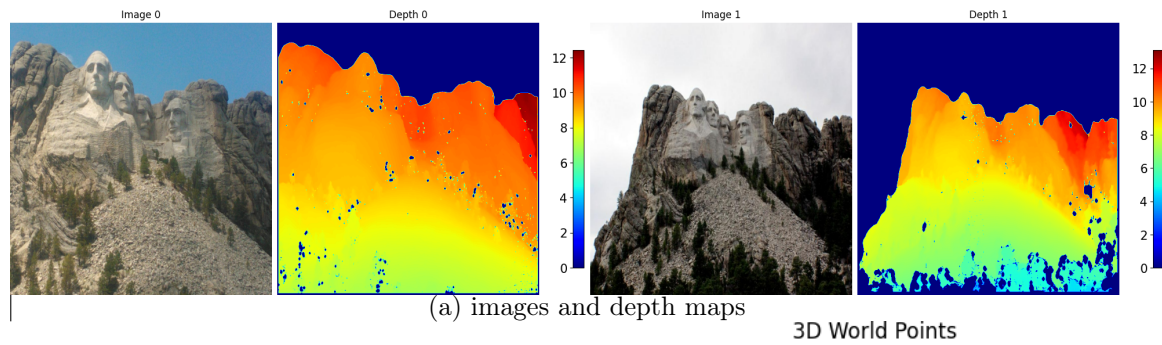


Figure 9: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 0

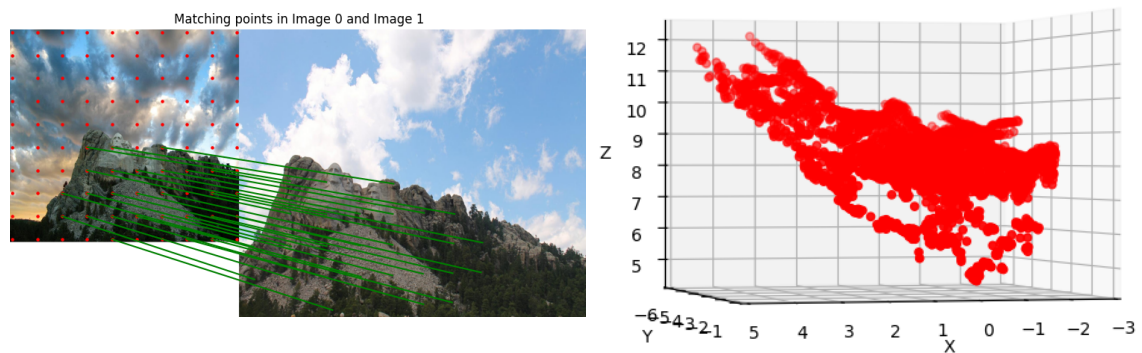
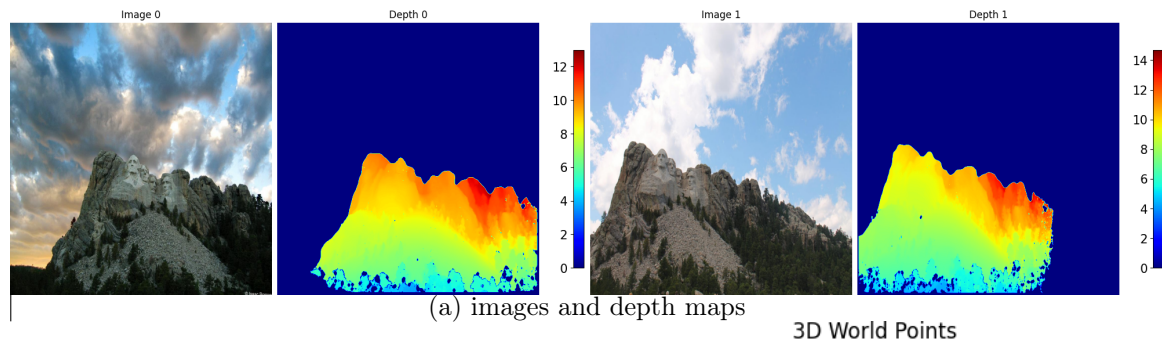


Figure 10: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 1

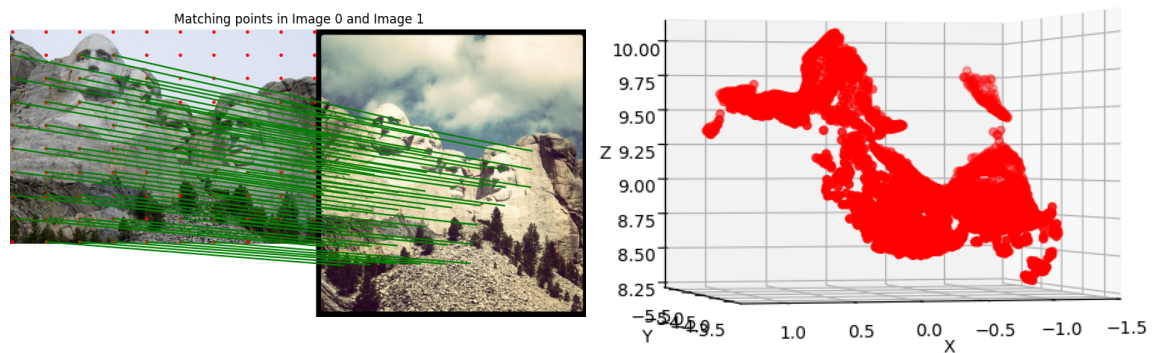
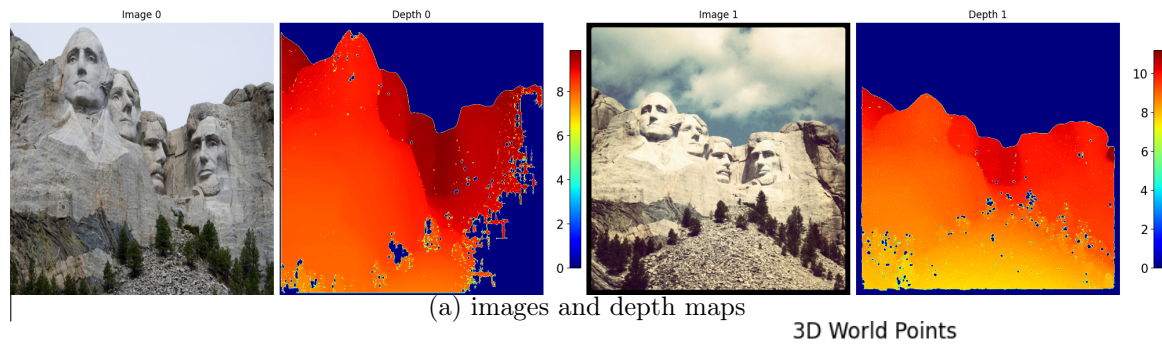


Figure 11: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 2

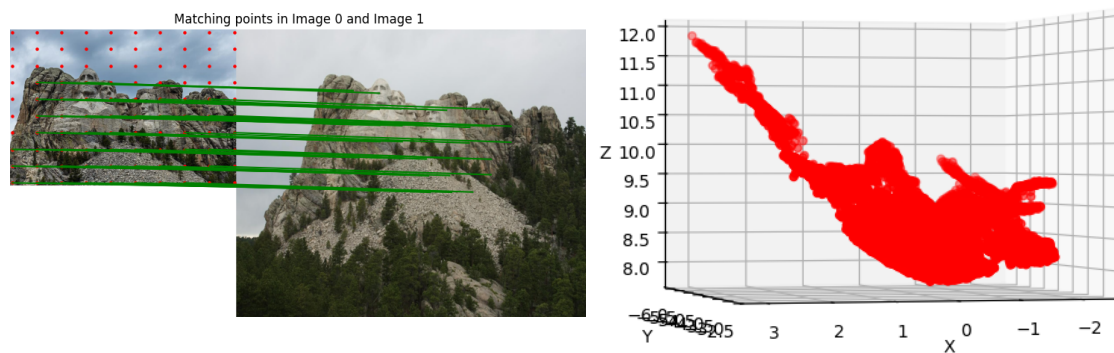
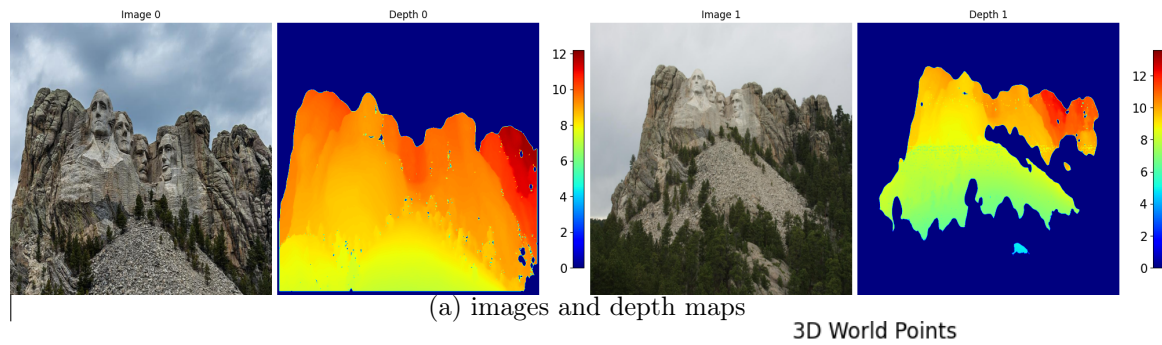


Figure 12: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 3

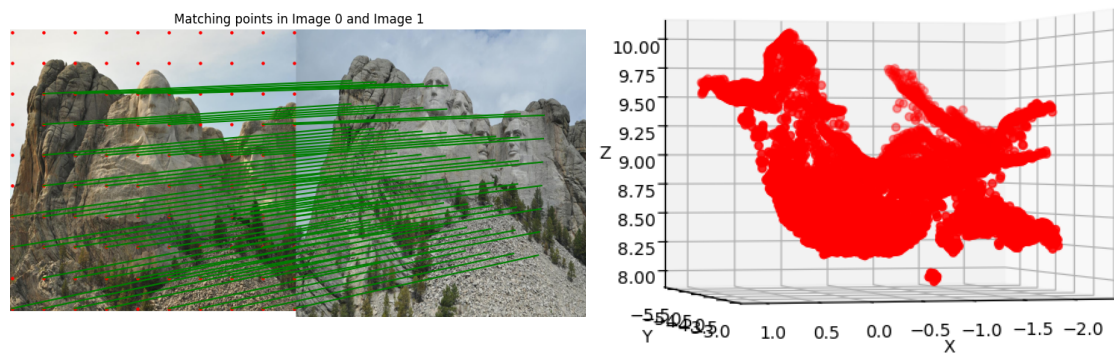
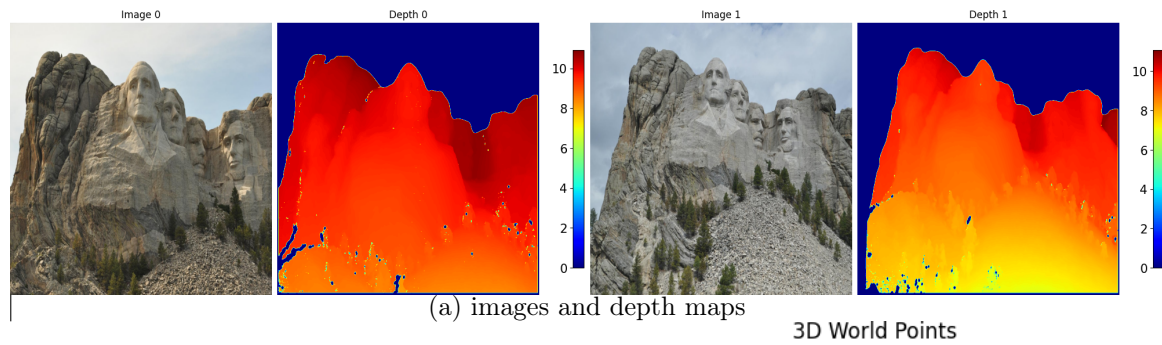


Figure 13: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 4

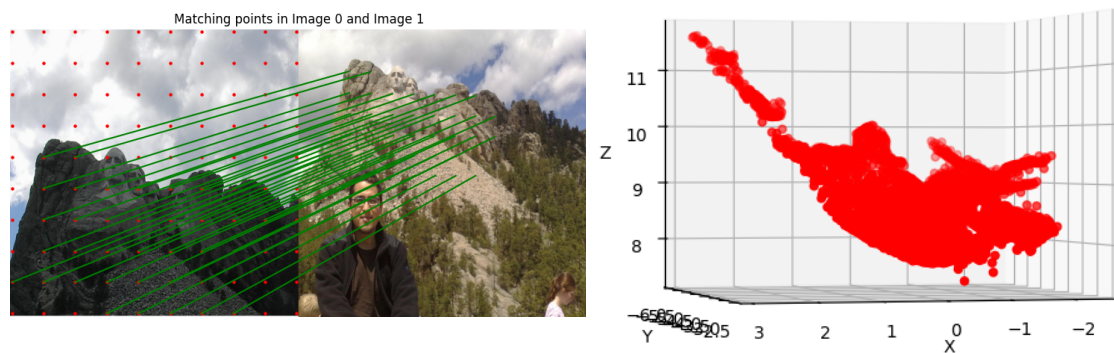
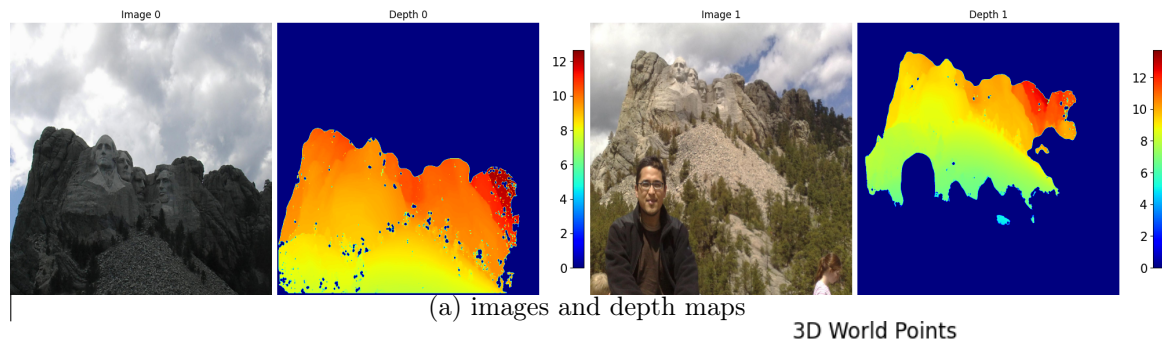


Figure 14: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 5

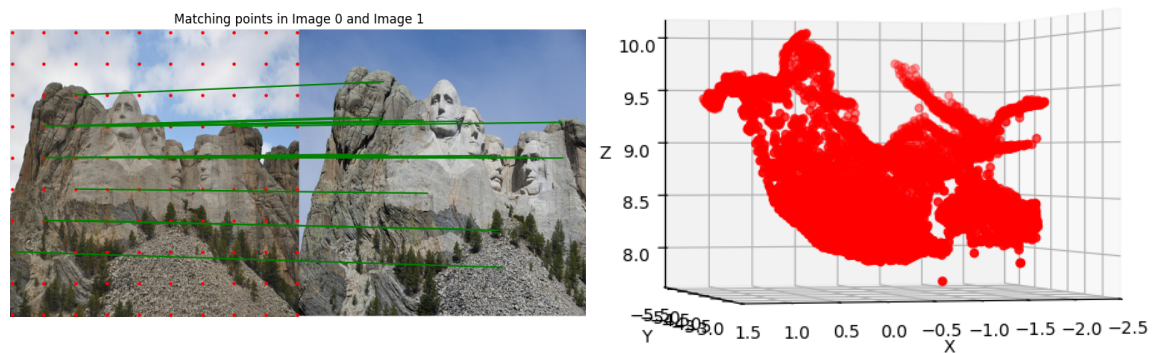
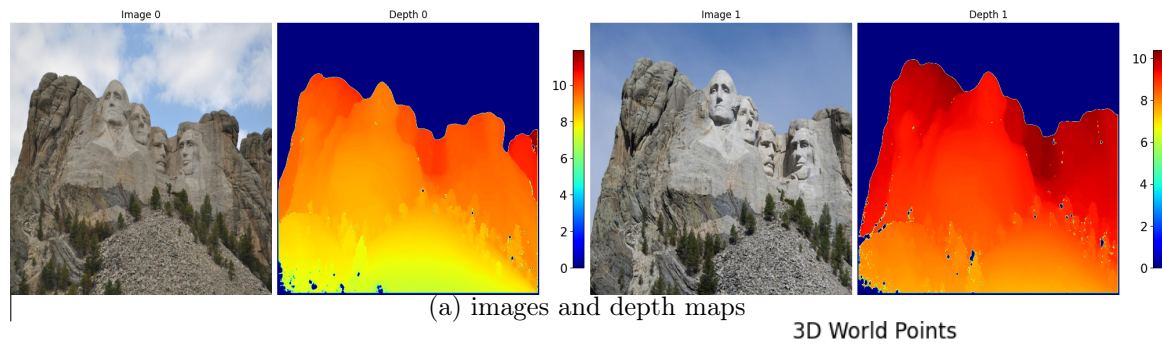


Figure 15: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 6

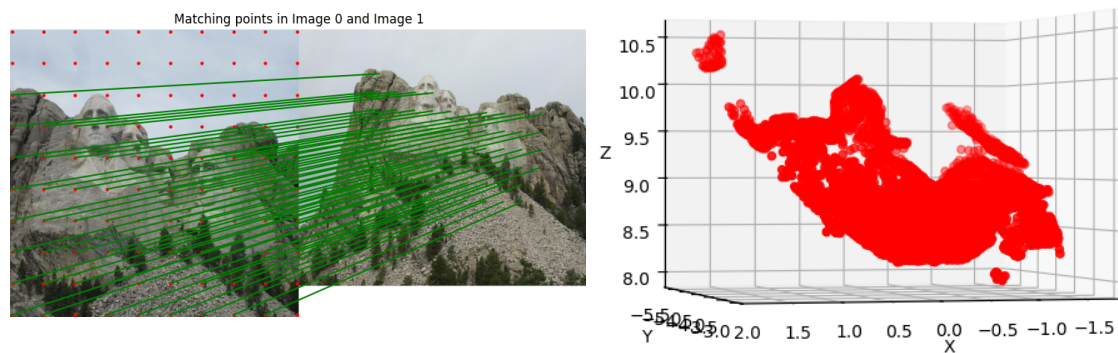
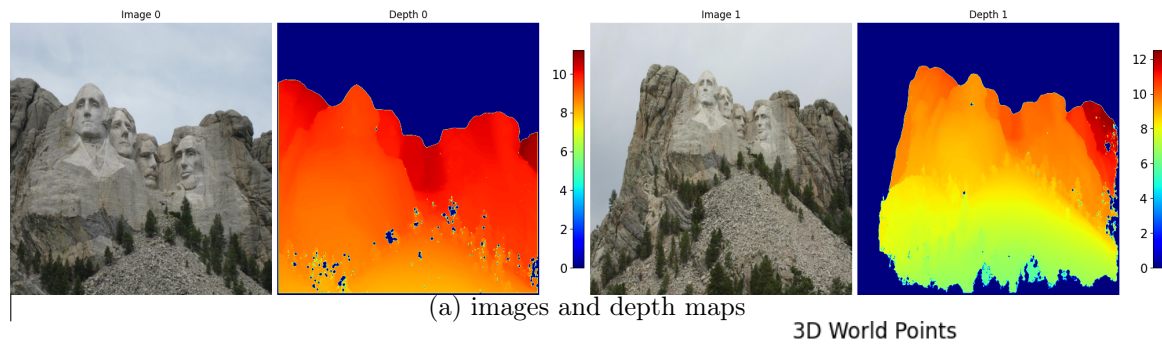


Figure 16: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 7

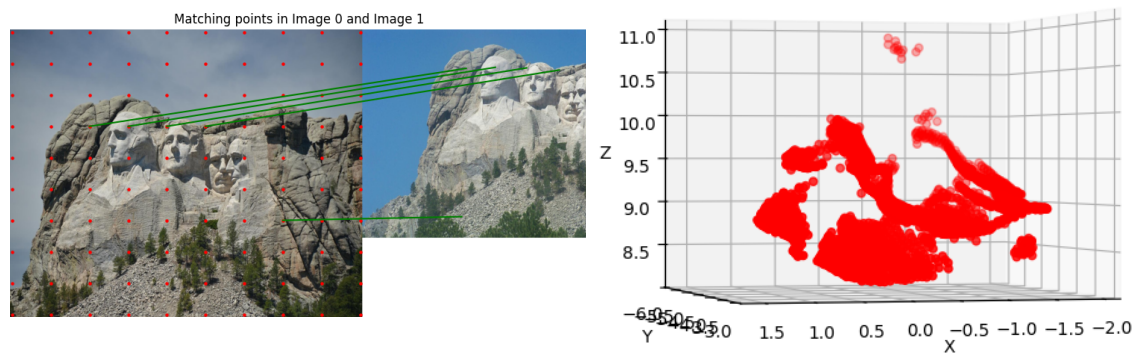
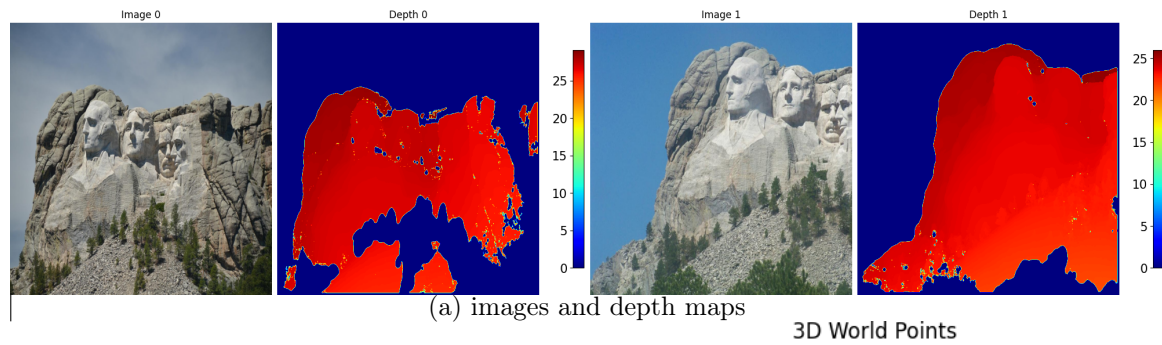


Figure 17: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 8

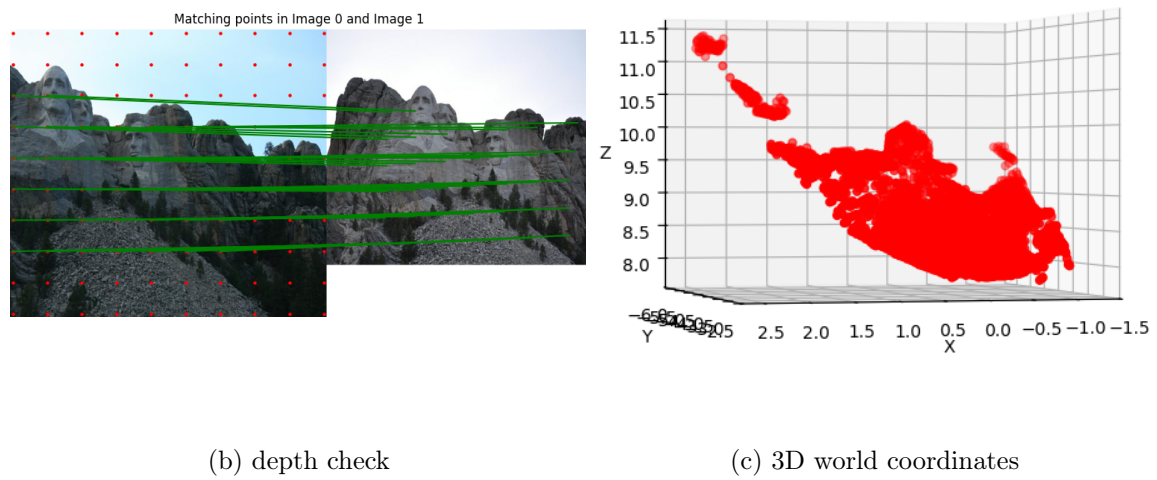
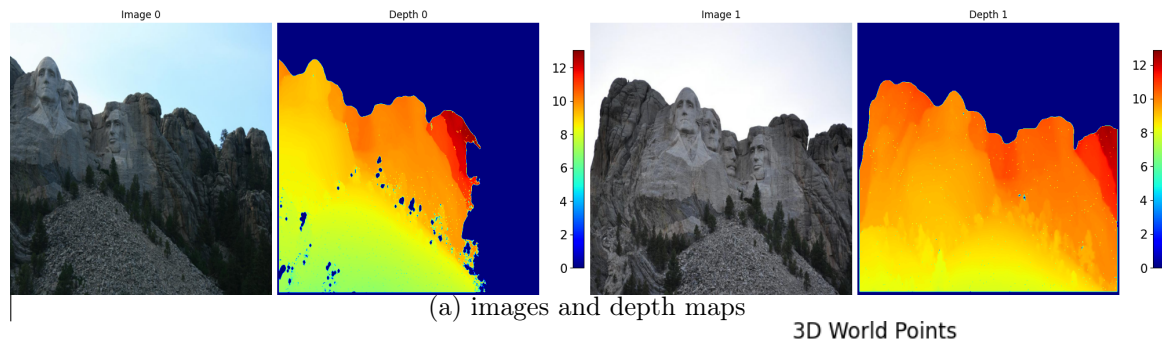


Figure 18: Camera images with their depth maps, output of dense map using depth check and 3D scattering of world coordinates for pair 9

```
15 import matplotlib as mpl
16 import plotly.graph_objects as go
17 from matplotlib.patches import ConnectionPatch
18 from mpl_toolkits.mplot3d import proj3d
19 import numpy as np
20 import math
21
22 import os
23 import glob
24 import time
25 import torch
26 from models.matching import Matching
27 from operator import itemgetter
28 from models.utils import (AverageTimer, VideoStreamer,
29                           make_matching_plot_fast, frame2tensor)
30
31
32 from sklearn.svm import SVC, LinearSVC
33 from sklearn.metrics import accuracy_score, confusion_matrix
34 import seaborn as sns
35
36 torch.set_grad_enabled(False)
37 import matplotlib
38
39
40
41
42 # Helper functions..
43
44 def read_image_from_path(image_path):
45     """Reads images from the subdir"""
46     img_bgr = cv2.imread(image_path)
47     # Convert the image from BGR to RGB format
48     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
49     return img_rgb
50
51
52 def read_coords(img, num_pts=1):
53     # Display the image
54     if matplotlib.get_backend() != 'tkagg':
55         matplotlib.use('TkAgg')
56     plt.imshow(img)
57     plt.title(f"Click on {num_pts} points to get their coordinates")
58
59     try:
60         coords = plt.ginput(num_pts, timeout=100)
61         print(f"Coordinates selected: {coords}")
62         coords = np.array(coords)
63     except Exception as e:
64         coords = None
65         print(f"Error occurred: {e}. Did you close the window accidentally?"
66             )
67     plt.close()
68     return coords
69
70
71 def manual_correspondences(img1, img2, num_pts=8):
72     pts_v1 = read_coords(img1, num_pts)
73     pts_v2 = read_coords(img2, num_pts)
```

```

73
74     assert (pts_v1 is not None) and (pts_v2 is not None), print(f"Points not
75         recorded..!")
76     return pts_v1, pts_v2
77
78 def plot_corresponding_keypoints(img1, img2, kp1, kp2):
79     """
80     Plots two images side by side and draws lines between corresponding
81         keypoints.
82
83     Parameters:
84     - img1, img2: The two input images to be plotted side by side.
85     - kp1, kp2: The corresponding keypoints in the form of lists of tuples
86         [(y1, x1), (y2, x2), ...].
87     """
88     # Combine both images horizontally to create a side-by-side view
89     combined_img = np.hstack((img1, img2)) # Horizontally stack img1 and
90         img2
91
92     # Create a figure for plotting
93     # fig, ax = plt.subplots(figsize=(12, 6))
94     plt.imshow(combined_img)
95     # ax.axis('off') # Hide axes for better visual appeal
96
97     # Plot corresponding keypoints and draw lines between them
98     for (x1, y1), (x2, y2) in zip(kp1, kp2):
99         color = [random.random() for _ in range(3)] # Random color for each
100             line
101
102         # Plot keypoints on the left image (img1)
103         plt.scatter(x1, y1, color=color, s=50, marker='o')
104
105         # Plot keypoints on the right image (img2) with an offset for the x-
106             coordinate
107         plt.scatter(x2 + img1.shape[1], y2, color=color, s=50, marker='o')
108         # Offset by width of img1
109
110         # Draw a line between the corresponding keypoints
111         plt.plot([x1, x2 + img1.shape[1]], [y1, y2], color=color, lw=1) #
112             Line between the two keypoints
113
114     # plt.show()
115
116 import scipy.linalg
117
118 class Rectifier:
119     def __init__(self, images_dir, img1_pts, img2_pts, save_img_pts=False)
120         -> None:
121         self.img1_name = 'image-1.jpg'
122         self.img2_name = 'image-2.jpg'
123
124         self.images_dir = images_dir
125
126         img1 = read_image_from_path(os.path.join(self.images_dir, self.
127             img1_name))

```

```

121         img2 = read_image_from_path(os.path.join(self.images_dir, self.
122             img2_name))
123
124         self.img1 = img1
125         self.img2 = img2
126
127         self.x = img1_pts
128         self.x_prime = img2_pts
129         self.num_pts = img1_pts.shape[0]
130
131         height, width, _ = self.img1.shape
132         center = np.array([width//2, height//2])[None,:]
133         self.T, self.T_prime = self.get_pre_conditioning_matrices()
134         # self.x = self.x - center
135         # self.x_prime = self.x_prime - center
136         if save_img_pts:
137             self.save_img_with_pts()
138
139     def get_pre_conditioning_matrices(self):
140         """Isotropic scalling to pre-condition points."""
141         img1_h, img1_w, _ = self.img1.shape
142         pixels_x, pixels_y = np.meshgrid(np.arange(img1_w), np.arange(img1_h))
143         dom_pixels = np.stack([pixels_x.flatten(), pixels_y.flatten()], axis=1)
144
145         T = Rectifier.compute_pre_condtioning_transform(dom_pixels)
146
147         img2_h, img2_w, _ = self.img2.shape
148         pixels_x, pixels_y = np.meshgrid(np.arange(img2_w), np.arange(img2_h))
149         range_pixels = np.stack([pixels_x.flatten(), pixels_y.flatten()], axis=1)
150
151         T_prime = Rectifier.compute_pre_condtioning_transform(range_pixels)
152         return T, T_prime
153
154     @staticmethod
155     def compute_pre_condtioning_transform(points_2d):
156         # compute centroid
157         centroid = np.mean(points_2d, axis=0)
158         # avg distance of point from centroid
159         d = np.sqrt(np.sum((points_2d - centroid[None, :])**2, axis=1))
160         d_avg = np.mean(d)
161
162         # scale factor to set avg distance to sqrt(2)
163         s = np.sqrt(2) / d_avg
164
165         # pre-conditioning matrix
166         T = np.eye(3)
167         T[0,0] = s
168         T[1,1] = s
169         T[:2, -1] = -s*centroid
170         return T
171
172
173     def save_img_with_pts(self):
174         """Save images with points to disk"""

```

```

175         output_name = f'img-pts-{self.img1_name}'
176         Rectifier.add_pts_to_img(self.img1, self.x)
177         self.save_fig(output_name)
178
179         output_name = f'img-pts-{self.img2_name}'
180         Rectifier.add_pts_to_img(self.img2, self.x_prime)
181         self.save_fig(output_name)
182
183
184
185
186     def save_fig(self, output_name):
187
188         filepath = os.path.join(self.images_dir, output_name)
189         plt.savefig(filepath)
190         print(f"saved to path: {filepath}")
191         plt.close()
192
193
194     @staticmethod
195     def add_pts_to_img(img, pts):
196         pts = pts.astype(np.int32)
197         pts_color = (255,0,0)
198         img = img.copy()
199         for i in range(pts.shape[0]):
200             x,y = pts[i]
201             cv2.circle(img, (x, y), radius=5, color=pts_color, thickness=-1)
202             cv2.putText(img, f"{i+1}", (x - 10, y-10), fontFace=cv2.
203                 FONT_HERSHEY_SIMPLEX,
204                 fontScale=0.8, color=pts_color, thickness=3)
205             title = f"image with recorded pts"
206             plt.imshow(img)
207             plt.title(title)
208
209     @staticmethod
210     def get_coefficient_vector(x, x_prime):
211         """Return the coefficients of linear equations,
212         for the given .
213
214         Args:
215             x: pt on 1st image
216             x: pt on 2nd image
217         """
218         return np.array([x_prime[0]*x[0], x_prime[0]*x[1], x_prime[0],
219             x_prime[1]*x[0], x_prime[1]*x[1], x_prime[1],
220             x[0], x[1], 1])
221
222     def get_coefficient_matrix(self, x_pts, x_prime_pts):
223         """For the given data points, get the coefficients matrix
224         A for homogeneous system of equations.
225         """
226         V = []
227         for i in range(self.num_pts):
228             V_i = Rectifier.get_coefficient_vector(x_pts[i], x_prime_pts[i])
229             V.append(V_i[None,:])
230         return np.concat(V, axis=0)
231
232

```



```

233 @staticmethod
234 def lst_square(A):
235     """Returns linear least square solution to homogeneous
236     system of linear equations,  $Ah = 0$ .
237     Since  $\text{Rank}(A) = n-1$ , solution is the eigen vector
238     corresponding to the smallest singular value of  $A$ 
239     i.e. last row of  $vh$ .
240     """
241     u, s, vh = np.linalg.svd(A)
242     return vh[-1]
243
244 @staticmethod
245 def pre_condition_points(points_2d, T):
246     """Isotropic scalling to pre-condition points."""
247
248     points_hc = np.concat([points_2d, np.ones(points_2d.shape[0])[:, None]
249                             ], axis=1)
249     points_norm = np.matmul(T, points_hc.T)
250     return points_norm.T
251
252 def get_initial_F(self, precondition=True):
253
254     if precondition:
255         x_norm = Rectifier.pre_condition_points(self.x, self.T)
256         x_prime_norm = Rectifier.pre_condition_points(self.x_prime, self
257             .T_prime)
258
259         A = self.get_coefficient_matrix(x_norm, x_prime_norm)
260         F = Rectifier.lst_square(A).reshape(3,3)
261
262         F = Rectifier.condition_F(F)
263         if precondition:
264             F_tilde = np.matmul(F, self.T)
265             F = np.matmul(np.linalg.pinv(self.T_prime), F_tilde)
266             F /= F[-1,-1]
267         return F
268
269 @staticmethod
270 def compute_epipolar_pts(F):
271     e = Rectifier.lst_square(F)
272     e /= e[-1]
273     e_prime = Rectifier.lst_square(F.T)
274     e_prime /= e_prime[-1]
275     return e, e_prime
276
277 def canonical_cameras(self, F, e_prime):
278
279     P = np.concatenate([np.eye(3), np.zeros((3,1))], axis=1)
280     e_prime_cross = Rectifier.cross_prod_matrix(e_prime)
281     P_prime = np.concatenate([e_prime_cross@F, e_prime[:, None]], axis
282         =1)
283     return P, P_prime
284
285 @staticmethod
286 def condition_F(F):
287     """Condition F to make it rank-2"""
288     u, s, vh = np.linalg.svd(F)
289     s[-1] = 0
290     F = u@(s*np.eye(3))@vh

```

```

289         return F
290
291     @staticmethod
292     def cross_prod_matrix(e):
293         return np.array([[0, -e[2], e[1]],
294                         [e[2], 0, -e[0]],
295                         [-e[1], e[0], 0]])
296
297
298     @staticmethod
299     def get_world_pts(x, x_prime, P, P_prime):
300         A = np.zeros((4,4))
301         A[0] = x[0]*P[2] - P[0]
302         A[1] = x[1]*P[2] - P[1]
303         A[2] = x_prime[0]*P_prime[2] - P_prime[0]
304         A[3] = x_prime[1]*P_prime[2] - P_prime[1]
305
306         X_world = Rectifier.lst_square(A)
307         X_world /= X_world[-1]
308         return X_world[:-1]
309
310
311     def compute_residuals(self, params):
312
313         P = np.concatenate([np.eye(3), np.zeros((3,1))], axis=1)
314         P_prime = params[:12].reshape(3,4)
315
316         x_hat = []
317         x_prime_hat = []
318         residuals = []
319         for i in range(self.num_pts):
320             world_x = params[12+3*i:12+3*(i+1)]
321
322             x_hat = Rectifier.get_img_pt(P, world_x)
323             x_prime_hat = Rectifier.get_img_pt(P_prime, world_x)
324             residuals.append(self.x[i] - x_hat)
325             residuals.append(self.x_prime[i] - x_prime_hat)
326
327
328         return np.array(residuals).flatten()
329
330     @staticmethod
331     def get_img_pt(P, X):
332         X = np.concatenate([X, np.ones(1)])
333         x = P@X
334         x /= x[-1]
335         return x[:-1]
336
337
338
339     def get_refined_estimates(self, P, P_prime):
340
341         params = np.zeros(12 + 3*(self.num_pts))
342         params[:12] = P_prime.reshape(-1)
343         for i in range(self.num_pts):
344             world_x = Rectifier.get_world_pts(self.x[i], self.x_prime[i], P,
345                                               P_prime)
346             params[12+3*i:12+3*(i+1)] = world_x[:3]

```

```

347     params = scipy.optimize.least_squares(self.compute_residuals, params
348     , method='trf').x
349     P_prime = params[:12].reshape(3,4)
350     e_prime = P_prime[:,-1]
351
352     F = Rectifier.cross_prod_matrix(e_prime)@P_prime[:,:3]
353     F /= F[-1,-1]
354     return F, P_prime
355
356 def compute_H_prime(self, e_prime):
357
358     e_prime /= e_prime[-1]
359     height, width, _ = self.img2.shape
360     x0 = width/1
361     y0 = height/1
362
363     T1 = np.eye(3)
364     T1[0,2] = -x0
365     T1[1,2] = -y0
366
367     theta = np.arctan(-(e_prime[1]-y0)/(e_prime[0]-x0))
368     R = np.eye(3)
369     R[0,0] = np.cos(theta)
370     R[0,1] = -np.sin(theta)
371     R[1,0] = np.sin(theta)
372     R[1,1] = np.cos(theta)
373
374     f = np.abs((e_prime[0] - x0)*np.cos(theta) - (e_prime[1] - y0)*np.
375             sin(theta))
376
377     G = np.eye(3)
378     G[2,0] = -1/f
379
380     T2 = np.eye(3)
381     T2[0,2] = x0
382     T2[1,2] = y0
383
384     H_prime = T2@G@R@T1
385
386     return H_prime/H_prime[-1,-1]
387
388 def compute_H(self, P, P_prime, H_prime):
389
390     P_pinv = P.T @ np.linalg.inv(P @ P.T)
391     M = P_prime @ P_pinv
392     H0 = H_prime @ M
393
394     H0 /= H0[-1,-1]
395
396     # Transform points
397     x_hat = Rectifier.transform_pixel_coordinate(H0, self.x)
398     x_prime_hat = Rectifier.transform_pixel_coordinate(H_prime, self.
399               x_prime)
400
401     # Solve for Ha
402     A = np.ones((x_hat.shape[0], 3))
403     A[:, :-1] = x_hat
404     b = x_prime_hat[:, 0]

```

```

403     # Solve  $Ax - b = 0$  using the left pseudo-inverse
404     a = (np.linalg.inv(A.T @ A) @ A.T) @ b
405     Ha = np.eye(3)
406     Ha[0, :] = a
407
408     print(f'Ha:\n{Ha}')
409
410     # Compute final H
411     H = Ha @ H0
412     return H, H0
413
414
415     @staticmethod
416     def transform_pixel_coordinate(H, pts):
417         # pts = np.random.randn(8,2)
418         hc_coords = np.concatenate([pts, np.ones(pts.shape[0])[:, None]],
419                                     axis=1).transpose()
420
421         transformed_hc = np.matmul(H, hc_coords) # gives me (3,n)
422         transformed_hc /= transformed_hc[-1]#[None, :]
423         return transformed_hc[:-1].transpose()
424
425
426 class EdgeDetector:
427     def __init__(self, rect_img1, rect_img2) -> None:
428         self.rect_img1 = rect_img1
429         self.rect_img2 = rect_img2
430         self.canny_th1 = 0.4
431         self.canny_th2 = 0.6
432         self.kp1 = None # Keypoints from the first image
433         self.kp2 = None # Keypoints from the second image
434         self.ssd_scores = None # SSD scores for the keypoints
435
436     def canny_edges(self, kernel_size=5, num_rows=3):
437         # Convert images to grayscale and apply Canny edge detection
438         img1_gray = cv2.cvtColor(self.rect_img1, cv2.COLOR_RGB2GRAY)
439         img2_gray = cv2.cvtColor(self.rect_img2, cv2.COLOR_RGB2GRAY)
440
441         edges1 = cv2.Canny(img1_gray, self.canny_th1, self.canny_th2,
442                             apertureSize=7, L2gradient=True)
443         edges2 = cv2.Canny(img2_gray, self.canny_th1, self.canny_th2,
444                             apertureSize=7, L2gradient=True)
445
446         self.kp1, self.kp2, self.ssd_scores = self.compute_ssd(
447             edges1, edges2, kernel_size=kernel_size, num_rows=num_rows
448         )
449
450     def compute_ssd(self, edges1, edges2, kernel_size=5, num_rows=3):
451         """Searches for matching key points only in 3 rows"""
452         # Assuming edges1 and edges2 are binary edge maps (True for edges,
453         # False for non-edges)
454         kernel_half = kernel_size // 2
455
456         kp1 = [] # Keypoints from the first image
457         kp2 = [] # Keypoints from the second image
458         ssd_scores = [] # Corresponding SSD scores

```

```

458         for i in range(kernel_half+num_rows//2, edges1.shape[0] -
459                         kernel_half - num_rows//2):
460             for j in range(kernel_half, edges1.shape[1] - kernel_half):
461                 if edges1[i, j]:
462                     # restricts search to only within the mask
463                     ssd_values = []
464                     # search for match only in 3 rows
465                     for x in range(i-num_rows//2, i+1+num_rows//2):
466                         for y in range(kernel_half, edges2.shape[1] -
467                                     kernel_half):
468                             if edges2[x,y]:
469                                 # print(x,y)
470                                 region1 = edges1[i-kernel_half:i+kernel_half
471                                                +1, j-kernel_half:j+kernel_half+1]
472                                 region2 = edges2[x-kernel_half:x+kernel_half
473                                                +1, y-kernel_half:y+kernel_half+1]
474
475                                 # print(region1.shape)
476                                 # print(region2.shape)
477                                 ssd_value = np.sum((region1 - region2) ** 2)
478                                 ssd_values.append(((j, i), (y, x), ssd_value
479                                                    ))
480
481                     min_ssd = min(ssd_values, key=lambda x: x[2]) # Sort by
482                             SSD value
483                     kp1.append(min_ssd[0])
484                     kp2.append(min_ssd[1])
485                     ssd_scores.append(min_ssd[2])
486
487     return np.array(kp1), np.array(kp2), np.array(ssd_scores)
488
489
490 def get_top_keypoints(self, top_percentage=10, max_points=100):
491
492     scores = self.ssd_scores
493     num_points = len(scores)
494
495     # Compute the number of top points based on the given percentage or
496     # max_points
497     num_top_points = min(int(num_points * top_percentage / 100),
498                           max_points)
499
500     # Sort the keypoints by SSD score in descending order
501     sorted_indices = sorted(range(len(scores)), key=lambda i: scores[i],
502                             reverse=False)
503
504     # Select the top keypoints and their scores
505     top_kp1 = np.array([self.kp1[i] for i in sorted_indices[:
506                             num_top_points]])
507     top_kp2 = np.array([self.kp2[i] for i in sorted_indices[:
508                             num_top_points]])
509     top_ssd_scores = np.array([scores[i] for i in sorted_indices[:
510                             num_top_points]])
511
512     return top_kp1, top_kp2, top_ssd_scores

```

```

505
506 class WorldProjector:
507     def __init__(self, left_image, right_image, x, x_prime, P, P_prime):
508
509         self.left_image = left_image
510         self.right_image = right_image
511         self.x = x
512         self.x_prime = x_prime
513         self.P = P
514         self.P_prime = P_prime #
515
516         self.num_pts = self.x.shape[0]
517         self.corner_ids = np.array([0, 1, 14,15], dtype=np.int32)
518
519
520     @staticmethod
521     def get_world_pts(x, x_prime, P, P_prime):
522         A = np.zeros((4,4))
523         A[0] = x[0]*P[2] - P[0]
524         A[1] = x[1]*P[2] - P[1]
525         A[2] = x_prime[0]*P_prime[2] - P_prime[0]
526         A[3] = x_prime[1]*P_prime[2] - P_prime[1]
527
528         X_world = Rectifier.lst_square(A)
529         X_world /= X_world[-1]
530         return X_world[:-1]
531
532     @staticmethod
533     def get_img_pt(P, X):
534         X = np.concatenate([X, np.ones(1)])
535         x = P@X
536         x /= x[-1]
537         return x[:-1]
538
539
540     def get_refined_world_pts(self):
541
542         params = np.zeros(3*(self.num_pts))
543         for i in range(self.num_pts):
544             world_x = Rectifier.get_world_pts(self.x[i], self.x_prime[i], P,
545                 P_prime)
546             params[3*i:3*(i+1)] = world_x[:3]
547
548         params = scipy.optimize.least_squares(self.compute_residuals, params
549             , method='trf').x
550         world_pts = []
551         for i in range(self.num_pts):
552             world_pts.append(params[3*i:3*(i+1)])
553
554         return np.array(world_pts)
555
556     def compute_residuals(self, params):
557         """Params are the world points."""
558         x_hat = []
559         x_prime_hat = []
560         residuals = []
561         for i in range(self.num_pts):
562             world_x = params[3*i:3*(i+1)]

```



```

562         x_hat = Rectifier.get_img_pt(self.P, world_x)
563         x_prime_hat = Rectifier.get_img_pt(self.P_prime, world_x)
564         residuals.append(self.x[i] - x_hat)
565         residuals.append(self.x_prime[i] - x_prime_hat)
566
567     return np.array(residuals).flatten()
568
569
570
571 def plot(self, view_elevation=10, view_azimuth=30):
572     """
573     Plot the 3D points and their corresponding projections on the 2D
574     images.
575     """
576     world_pts = self.get_refined_world_pts()
577     # points_3d = self.project_to_3d()
578
579     # 1. 3D Plot of world points
580     fig = plt.figure(figsize=(18, 6))
581     ax_3d = fig.add_subplot(132, projection='3d')
582     ax_3d.scatter(world_pts[:, 0], world_pts[:, 1], world_pts[:, 2],
583                  color='r', marker='o')
584     ax_3d.set_title("3D World Points")
585     ax_3d.set_xlabel('X')
586     ax_3d.set_ylabel('Y')
587     ax_3d.set_zlabel('Z')
588     # Set view angle
589     ax_3d.view_init(elev=view_elevation, azim=view_azimuth) # 30
590     degrees elevation, 45 degrees azimuth
591
592     # 2. Plot the 2D projections on the left image
593     ax_left = fig.add_subplot(131)
594     ax_left.imshow(self.left_image)
595     ax_left.set_title("Left Image Projections")
596     ax_left.scatter(self.x[:, 0], self.x[:, 1], color='g', s=50, marker=
597                    'x')
598
599     # 3. Plot the 2D projections on the right image
600     ax_right = fig.add_subplot(133)
601     ax_right.imshow(self.right_image)
602     ax_right.set_title("Right Image Projections")
603     ax_right.scatter(self.x_prime[:, 0], self.x_prime[:, 1], color='b',
604                     s=50, marker='x')
605
606     # Connect the 3D points to their projections
607     left_img_lines = []
608     right_img_lines = []
609     world_lines = []
610     colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:gray',
611              'tab:cyan', 'tab:brown', 'tab:pink']
612     # possible_colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red',
613                        'tab:gray', 'tab:cyan']
614     for ii in range(len(self.corner_ids)):
615         for jj in range(ii+1, len(self.corner_ids)):
616             corner1 = self.corner_ids[ii]
617             corner2 = self.corner_ids[jj]
618             # if corner1 != corner2:
619             x1, y1 = self.x[corner1]
620             x2, y2 = self.x[corner2]

```

```

614         left_img_lines.append([[x1, x2], [y1, y2]])
615         x1, y1 = self.x_prime[corner1]
616         x2, y2 = self.x_prime[corner2]
617         right_img_lines.append([[x1, x2], [y1, y2]])
618         x1, y1, z1 = world_pts[corner1]
619         x2, y2, z2 = world_pts[corner2]
620         world_lines.append([[x1, x2], [y1, y2], [z1, z2]])
621         # colors.append(np.random.rand(3,))
622     # colors = possible_colors
623     for i, line in enumerate(left_img_lines):
624         ax_left.plot(line[0], line[1], color=colors[i], linewidth=3)
625
626     for i, line in enumerate(right_img_lines):
627         ax_right.plot(line[0], line[1], color=colors[i], linewidth=3)
628
629     for i, line in enumerate(world_lines):
630         ax_3d.plot(line[0], line[1], line[2], color=colors[i], linewidth
631                    =3)
632
633     # adding lines across subplots
634     for c_id in self.corner_ids:
635         xyleft = self.x[c_id]
636         xyright = self.x_prime[c_id]
637         xyworld = proj3d.proj_transform(*world_pts[c_id][:3], ax_3d.
638                                         get_proj())[2]
639
640         con = ConnectionPatch(
641             xyA=xyleft, xyB=xyworld, coordsA=ax_left.transData, coordsB=
642             ax_3d.transData,
643             color='red', linestyle="--", linewidth=2
644         )
645         fig.add_artist(con)
646         con = ConnectionPatch(
647             xyA=xyworld, xyB=xyright, coordsA=ax_3d.transData, coordsB=
648             ax_right.transData,
649             color='red', linestyle="--", linewidth=2
650         )
651         fig.add_artist(con)
652     plt.tight_layout()
653
654
655
656
657
658
659
660
661     # Dense Stereo Map implementation..
662     class DSM:
663         def __init__(self, images_dir, window=(7,7)):
664             self.images_dir = images_dir
665             self.left_img = read_image_from_path(os.path.join(images_dir, 'im2.
666                                                         png'))
667             self.right_img = read_image_from_path(os.path.join(images_dir, 'im6.
668                                                         png'))

```

```

667         self.left_disp_map = DSM.adjust_disp_map(
668             read_image_from_path(os.path.join(images_dir, 'disp2.png'))
669         )
670         self.right_disp_map = DSM.adjust_disp_map(
671             read_image_from_path(os.path.join(images_dir, 'disp6.png'))
672         )
673
674         self.row_offset = window[0]//2
675         self.col_offset = window[1]//2
676
677     @staticmethod
678     def adjust_disp_map(disparity_map):
679         disparity_map = disparity_map.astype(np.float32)/4
680         return disparity_map.astype(np.uint8)
681
682     def get_bitvector(self, img, row, col):
683         img_window = img[row:row+2*self.row_offset+1, col:col+2*self.
684             col_offset+1]
685         wind_bitvector = img_window > img[row, col]
686         return wind_bitvector
687
688     def compute_left_disparity_map(self):
689
690         row_offset = self.row_offset
691         col_offset = self.col_offset
692
693         d_max = np.max(self.left_disp_map)
694
695         img_left_g = cv2.cvtColor(self.left_img, cv2.COLOR_RGB2GRAY)
696         img_right_g = cv2.cvtColor(self.right_img, cv2.COLOR_RGB2GRAY)
697
698         padded_left = np.pad(img_left_g, (row_offset, col_offset), mode='
699             reflect')
700         padded_right = np.pad(img_right_g, (row_offset, col_offset), mode='
701             reflect')
702
703         height, width = img_left_g.shape
704         disparity_map = np.zeros((height, width), dtype=np.int32)
705
706         for row in range(height):
707             for col in range(width):
708                 left_bitvector = self.get_bitvector(padded_left, row,
709                     col)
710                 data_costs = []
711                 for d in range(d_max):
712                     # print(d)
713                     d_col = col - d
714                     if d_col >= 0:
715                         right_bitvector = self.get_bitvector(
716                             padded_right, row, d_col)
717                         cost = np.bitwise_xor(left_bitvector,
718                             right_bitvector).sum().item()
719                         data_costs.append((d, cost))
720                     # print(d_col, cost)

```

```

720         min_cost = min(data_costs, key=lambda x: x[1]) # Sort
721             by data cost
722         disparity_map[row, col] = min_cost[0]
723         # print(min_cost)
724         # break
725
726         ground_truth = cv2.cvtColor(self.left_disp_map, cv2.COLOR_RGB2GRAY)
727         non_black_mask = ground_truth > 0
728         diff = np.abs(ground_truth - disparity_map)
729         acc = np.count_nonzero(diff[non_black_mask] < 2) / np.count_nonzero(
730             non_black_mask)
731         print(f"Accuracy: {acc}")
732
733         error_mask = np.zeros_like(non_black_mask)
734         error_mask[diff < 2] = True
735         error_mask[np.bitwise_not(non_black_mask)] = False
736         return disparity_map.astype(np.uint8), error_mask.astype(np.uint8),
737             acc
738
739 if __name__ == "__main__":
740     # Task-1: Image rectification
741     images_dir = r'C:\Users\ahmedb\projects\computer-vision\images\hw9'
742
743     img1_pts = np.array([(208, 110), (493, 104), (211, 163), (498, 157),
744         (211, 209), (502, 198), (212, 255), (507, 241),
745         (219, 312), (360, 267), (514, 319), (221, 369),
746         (405, 516), (520, 369), (221, 574), (540, 557)])
747     img2_pts = np.array([(147, 132), (418, 103), (146, 182), (420, 154),
748         (144, 222), (422, 198), (144, 264), (423, 239),
749         (145, 313), (277, 270), (429, 316), (144, 370),
750         (308, 513), (433, 364), (135, 560), (443, 559)])
751
752     rectifier = Rectifier(images_dir, img1_pts, img2_pts)
753
754     plot_corresponding_keypoints(rectifier.img1, rectifier.img2, img1_pts,
755         img2_pts)
756     filepath = os.path.join(images_dir, 'images-with-manual-pts.jpg')
757     plt.savefig(filepath)
758     plt.close()
759     print(f"image saved to {filepath}")
760
761     F = rectifier.get_initial_F()
762     e, e_prime = Rectifier.compute_epipolar_pts(F)
763     P, P_prime = rectifier.canonical_cameras(F, e_prime)
764
765     F, P_prime = rectifier.get_refined_estimates(P, P_prime)
766     e, e_prime = Rectifier.compute_epipolar_pts(F)
767
768     print(f"e: {e}")
769     print(f"e_prime: {e_prime}")
770
771     H_prime = rectifier.compute_H_prime(e_prime)
772     H, H0 = rectifier.compute_H(P, P_prime, H_prime)
773     print(f"Hp: {H_prime}")
774     print(f"H: {H}")
775     print(f"H0: {H0}")

```

```
775 rectified_img1 = cv2.warpPerspective(rectifier.img1 , H, (600, 1200))
776 rectified_img2 = cv2.warpPerspective(rectifier.img2 , H_prime, (600,
777     1200))
778
779 trans_x = Rectifier.transform_pixel_coordinate(H, rectifier.x)
780 trans_x_prime = Rectifier.transform_pixel_coordinate(H_prime, rectifier.
    x_prime)
781 plot_corresponding_keypoints(rectified_img1, rectified_img2, trans_x,
    trans_x_prime)
782 filepath = os.path.join(images_dir, 'rectified-images-with-manual-pts.
    jpg')
783 plt.savefig(filepath)
784 plt.close()
785 print(f"image saved to {filepath}")
786
787
788 # Detecting key points
789 edge_detector = EdgeDetector(rectified_img1, rectified_img2)
790
791 # Apply Canny edge detection
792 edge_detector.canny_edges(kernel_size=21)
793
794 top_kp1, top_kp2, top_ssd_scores = edge_detector.get_top_keypoints(
    max_points=500)
795
796 plot_corresponding_keypoints(rectified_img1, rectified_img2, top_kp1
    [:200], top_kp2[:200])
797 filepath = os.path.join(images_dir, 'rectified-canny-pairs.jpg')
798 plt.savefig(filepath)
799 plt.close()
800 print(f"image saved to {filepath}")
801
802
803 img_kp1 = Rectifier.transform_pixel_coordinate(np.linalg.pinv(H) ,
    top_kp1)
804 img_kp2 = Rectifier.transform_pixel_coordinate(np.linalg.pinv(H_prime) ,
    top_kp2)
805
806 img_kp1 = np.concatenate([rectifier.x, img_kp1])
807 img_kp2 = np.concatenate([rectifier.x_prime, img_kp2])
808
809 projector = WorldProjector(rectifier.img1, rectifier.img2, img_kp1
    [:200], img_kp2[:200], P, P_prime)
810 # world_pts = projector.get_refined_world_pts()
811 projector.plot(45, 30)
812 filepath = os.path.join(images_dir, 'image-keypoints-projected-world-3d-
    view-1.jpg')
813 plt.savefig(filepath)
814 plt.close()
815 print(f"image saved to {filepath}")
816
817
818 projector.plot(60, 0)
819 filepath = os.path.join(images_dir, 'image-keypoints-projected-world-3d-
    view-2.jpg')
820 plt.savefig(filepath)
821 plt.close()
822 print(f"image saved to {filepath}")
```

```

823
824
825
826 # Task-3: Dense Stereo Map
827 images_dir = r'C:\Users\ahmedb\projects\computer-vision\images\hw9'
828
829 print(f"Task-3: Dense Stereo Map")
830 windows = [(11,11), (9,9)]
831 for i, window in enumerate(windows):
832     print(f"Running for window: {window}")
833     dsm = DSM(images_dir, window=window)
834
835     disparity_map, error_map, acc = dsm.compute_left_disparity_map()
836
837     plt.imshow(disparity_map, cmap='gray')
838     plt.title(f"window= {window}, Acc: {acc:.3f}")
839     filepath = os.path.join(images_dir, f'dense-stereo-map-wind{i+1}.jpg
840                             ')
841     plt.savefig(filepath)
842     plt.close()
843
844     plt.imshow(error_map, cmap='gray')
845     plt.title(f"error map for window={window}")
846     filepath = os.path.join(images_dir, f'stereo-error-map-wind{i+1}.jpg
847                             ')
848     plt.savefig(filepath)
849     plt.close()

```

Listing 1: Example Python code

5.2 Code for Task-4

```

1  import numpy as np
2  import pickle as pkl
3  import matplotlib.pyplot as plt
4  import h5py # for reading depth maps
5
6  """
7  A few notes on the scene_info dictionary:
8  - depth maps are stored as h5 files. Depth is the distance of the object
9    from the camera (ie Z coordinate in camera coordinates). The depth map
10   can contain invalid points (depth = 0) which correspond to points where
11   the depth could not be estimated.
12   - The intrinsics are stored as a 3x3 matrix.
13   - The poses [R,t] are stored as a 4x4 matrix to allow for easy
14     transformation of points from one camera to the other. The resulting
15     transformation matrix is a 4x4 matrix is of the form:
16     T = [[R, t]
17           [0, 1]] where R is a 3x3 rotation matrix and t is a 3x1 translation
18           vector.
19   """
20
21 DEPTH_THR = 0.1
22
23 def plot_image_and_depth(img0, depth0, img1, depth1, plot_name):
24     # Enable constrained layout for uniform subplot sizes
25     fig, ax = plt.subplots(1, 4, figsize=(20, 5), constrained_layout=True)

```



```

21 # Image 0
22 ax[0].imshow(img0, aspect='auto')
23 ax[0].set_title('Image 0')
24 ax[0].axis('off')
25
26 # Depth 0
27 im1 = ax[1].imshow(depth0, cmap='jet', aspect='auto')
28 ax[1].set_title('Depth 0')
29 ax[1].axis('off')
30 cbar1 = fig.colorbar(im1, ax=ax[1], shrink=0.8, aspect=20)
31 cbar1.ax.yaxis.set_ticks_position('left')
32 cbar1.ax.yaxis.set_label_position('left')
33 cbar1.ax.tick_params(labelsize=15)
34
35 # Image 1
36 ax[2].imshow(img1, aspect='auto')
37 ax[2].set_title('Image 1')
38 ax[2].axis('off')
39
40 # Depth 1
41 im2 = ax[3].imshow(depth1, cmap='jet', aspect='auto')
42 ax[3].set_title('Depth 1')
43 ax[3].axis('off')
44 cbar2 = fig.colorbar(im2, ax=ax[3], shrink=0.8, aspect=20)
45 cbar2.ax.yaxis.set_ticks_position('left')
46 cbar2.ax.yaxis.set_label_position('left')
47 cbar2.ax.tick_params(labelsize=15)
48
49 plt.savefig(plot_name, bbox_inches='tight', pad_inches=0)
50 plt.close()
51
52 if __name__ == "__main__":
53     scene_info = pickle.load(open('./data/scene_info/1589_subset.pkl', 'rb'))
54
55     for i_pair in range(len(scene_info)):
56         # print(scene_info[i_pair].keys())
57         # ['image0', 'image1', 'depth0', 'depth1', 'K0', 'K1', 'T0', 'T1', '
58         # overlap_score']
59         # print(scene_info[i_pair]['image0']) # path to image0
60         # print(scene_info[i_pair]['image1']) # path to image1
61         # print(scene_info[i_pair]['depth0']) # path to depth0
62         # print(scene_info[i_pair]['depth1']) # path to depth1
63         # print(scene_info[i_pair]['K0']) # intrinsic matrix of camera 0
64         # print(scene_info[i_pair]['K1']) # intrinsic matrix of camera 1
65         # print(scene_info[i_pair]['T0']) # pose matrix of camera 0 [4,4]
66         # print(scene_info[i_pair]['T1']) # pose matrix of camera 1 [4,4]
67         # print('-----')
68
69         # read images
70         img0 = plt.imread(scene_info[i_pair]['image0'])
71         img1 = plt.imread(scene_info[i_pair]['image1'])
72
73         # read depth
74         with h5py.File(scene_info[i_pair]['depth0'], 'r') as f:
75             depth0 = f['depth'][:]
76         with h5py.File(scene_info[i_pair]['depth1'], 'r') as f:
77             depth1 = f['depth'][:]

```

```

77
78     # check shapes
79     h0, w0 = img0.shape[: -1]
80     h1, w1 = img1.shape[: -1]
81     assert img0.shape[: -1] == depth0.shape, f"depth and image shapes do
        not match: {img0}, {depth0}"
82     assert img1.shape[: -1] == depth1.shape, f"depth and image shapes do
        not match: {img1}, {depth1}"
83
84     # plot image and depth
85     plot_name = f'./pics/image_and_depth_pair_{i_pair}.png'
86     plot_image_and_depth(img0, depth0, img1, depth1, plot_name)
87
88     #(1) make meshgrid of points in image 0
89     x = np.linspace(10, img0.shape[1]-10, 10) # ignore a border of 10
        pxls
90
91     # <Student code>
92     # meshgrid of x and y coordinates
93     y = np.linspace(10, img0.shape[0]-10, 10) # ignore a border of 10
        pxls
94     xx, yy = np.meshgrid(x,y)
95
96     # make homogeneous coordinates for points0 #[3, N]
97
98     # <Student code>
99     points0 = np.stack([xx.flatten(), yy.flatten(), np.ones_like(xx).
        flatten()], axis=0)
100
101     #(2) get depth values at points0
102     """
103     <Student code>
104     depth_values0 = ...
105     """
106     depth_values0 = depth0[points0[1].astype(np.uint16), points0[0].
        astype(np.uint16)]
107     # remove points with depth 0 (invalid points)
108     """
109     <Student code>
110     valid_points = ...
111     # mask points0 and depth_values0
112     """
113     mask = depth_values0 > 0
114     points0 = points0[:, mask]
115     depth_values0 = depth_values0[mask]
116
117     # (3) Find the 3D coordinates of these points in camera 0 frame
118     K0 = scene_info[i_pair]['K0'] # [3,3]
119     T0 = scene_info[i_pair]['T0'] # [4,4]
120
121     # <Student code>
122     # inverse of K0
123     alpha_x = K0[0,0]
124     s = K0[0,1]
125     x0 = K0[0,2]
126     alpha_y = K0[1,1]
127     y0 = K0[1,2]
128     K0_inv = np.eye(3)
129     K0_inv[0,0] = 1/alpha_x

```

```

130     K0_inv[0,1] = -s/(alpha_x*alpha_y)
131     K0_inv[0,2] = (s*y0 - alpha_y*x0)/(alpha_x*alpha_y)
132     K0_inv[1,1] = 1/alpha_y
133     K0_inv[1,2] = -y0/alpha_y
134     # convert points0 to camera coordinates
135     xyz_cam0 = K0_inv@points0
136     # normalize xyz_cam0 to set z = 1 (sanity check)
137     xyz_cam0 = xyz_cam0/xyz_cam0[-1]
138     # get the point at depth
139     xyz_cam0 = depth_values0*xyz_cam0
140     # make homogeneous coordinates [4,N]
141     xyz_cam0_hc = np.concatenate([xyz_cam0, np.ones((1, xyz_cam0.shape
142                                     [1]))], axis=0)
143     # convert to world frame [4,N]
144     xyz_world_hc = np.linalg.inv(T0)@xyz_cam0_hc
145
146     # (4) Transform these points to camera 1 frame
147     T1 = scene_info[i_pair]['T1']
148
149     # <Student code>
150     # transform points to camera 1 frame
151     xyz_cam1_hc = T1@xyz_world_hc
152     # convert to camera 1 coordinates
153     xyz_cam1 = xyz_cam1_hc[:-1]
154     # get z coordinates for depth check
155     estimated_depth_values1 = xyz_cam1[2]
156
157
158     # project to image 1
159     # <Student code>
160     K1 = scene_info[i_pair]['K1'] # [3,3]
161     points1 = K1@xyz_cam1 # [3, N]
162     # normalize by dividing by last row
163     points1 /= points1[-1] # [3, N]
164     # check if points1 are within image bounds
165     img1_mask = (points1[0] >= 0) & (points1[1] >= 0) & (points1[0] <
166                                     img1.shape[1]) & (points1[1] < img1.shape[0])
167     points1 = points1[:,img1_mask].astype(np.int16)
168     # get the depth values at these points using the depth map
169     true_depth_values1 = depth1[points1[1],points1[0]]
170
171
172     # (5) plot matching points in image 0 and image 1 with depth check
173     # such that the depth values match
174     fig, ax = plt.subplots(1, 1, figsize=(10, 5))
175
176     # Horizontally stack the images
177     combined_img = np.ones((max(img0.shape[0], img1.shape[0]), img0.
178                             shape[1] + img1.shape[1], 3), dtype=np.uint8) * 255
179     combined_img[:img0.shape[0], :img0.shape[1]] = img0
180     combined_img[:img1.shape[0], img0.shape[1]:] = img1
181
182     ax.imshow(combined_img, aspect='auto')
183     ax.scatter(xx, yy, c='r', s=5)
184     ax.set_title('Matching points in Image 0 and Image 1')
185     ax.axis('off')
186
187     # draw lines between matching points

```

```

185     for i in range(points1.shape[1]):
186         # if depth values match
187         if np.abs(estimated_depth_values1[i] - true_depth_values1[i]) <
            DEPTH_THR and true_depth_values1[i] != 0:
188             ax.plot([points0[0,i], points1[0, i] + img0.shape[1]], [
                points0[1,i], points1[1, i]], 'g')
189
190     plt.savefig(f'./pics/depth_check_pair_{i_pair}.png', bbox_inches='
        tight', pad_inches=0)
191     plt.close()
192     print(f"Done with pair {i_pair}")
193
194     # (6) Plot all 3D points for the pair
195
196     # <Student code>
197     #
198     # 1. 3D Plot of world points
199     fig = plt.figure(figsize=(6,6))
200     ax_3d = plt.subplot(projection='3d')
201     xyz_world = xyz_world_hc[:-1, img1_mask]
202     ax_3d.scatter(xyz_world[0, :], xyz_world[1, :], xyz_world[2, :],
        color='r', marker='o')
203     ax_3d.set_title("3D World Points")
204     ax_3d.set_xlabel('X')
205     ax_3d.set_ylabel('Y')
206     ax_3d.set_zlabel('Z')
207     # Set view angle
208     ax_3d.view_init(elev=0, azim=75) # 30 degrees elevation, 45 degrees
        azimuth
209
210     plt.savefig(f'./pics/world_points_3d_{i_pair}.png', bbox_inches='
        tight', pad_inches=0)
211     plt.close()
212     print(f"Done with world 3d {i_pair}")

```

Listing 2: Example Python code